جامعة الأميرة
نورة بنت عبدالرحمن
كلية علوم الحاسب والمعلومات

CS 111 - Lab

Programming Language 2

Computer Science Department

2026

Lab 1.2

Objects and Classes

Lab Objectives:

In this lab, a student will practice

• In this lab, the student will practice: • Declaring static variables and static methods • Using final instance variables • Passing objects to methods • Copy constructor • Creating and using an array of objects • Using Strings and wrapper classes	

Lab Exercise 1: Employee System

Based on the UML class diagram below, create a Java class named **Employee**.
The class should include all required instance variables, constructors, setters, and getters.
Then, write a test application to demonstrate the class capabilities.

Employee
- EMPLOYEE_ID : String - name : String - department : String - salary : double - employeeRef : String - totalEmployees : int
+ Employee(EMPLOYEE_ID : String, name : String, department : String, salary : double) + Employee(employee : Employee) + setName(name : String) : void + getName() : String + setDepartment(department : String) : void + getDepartment() : String + setSalary(salary : double) : void + getSalary() : double + displayInfo() : void + getTotalEmployees() : int + generateEmployeeRef() : String

Consider the following points:

1. Attributes

The class **Employee** must include the following instance variables:

- EMPLOYEE_ID
  - o Declare employeeId as a **final** instance variable.
  - o Assigned only once inside the constructor.
- **name**
  - o Stores the employee name.
- **department**
  - o Each employee belongs to one department only.
- **salary**
  - o Must always be greater than 0.
- **employeeRef**
  - o Generated using generateEmployeeRef().
- **totalEmployees**
  - o A variable that counts how many Employee objects have been created.

2. Constructors

- Implement all constructors shown in the UML diagram.
- Each constructor must increment totalEmployees.

3. Methods

• **setSalary(double salary)**
o If salary ≤ 0, display:

Invalid salary value

• **setName(String name)**
o Store the name in **uppercase**.

• **generateEmployeeRef()**
o The reference code is formed by:
▪ The **first two characters of the department name**
▪ The **last two characters of the employeeId**
o Concatenate without spaces and store the result in uppercase.

Example
• Department = Finance
• EmployeeId = EMP789
• Reference Code = **FI89**

Test Class Requirements

1. Create three **Employee** objects using different constructors.
2. Store the objects in an array of **Employee**.
3. Use a loop to display all employee information.
4. Print the total number of created employees.
5. Ask the user to enter a new salary for the second employee.
6. Convert the entered value to **Double**.
7. Update the salary using setSalary().
8. Print only the **name and salary** of the second employee.
9. Add a method named **searchEmployee(Employee[] employees, String employeeId)** in the test class:

   • Ask the user to enter an **employeeId**.
   • Search within the array using equals() or equalsIgnoreCase().

   If found:
   • Display: Employee found.

   • Print all employee information.

   If not found:
   • Display: Employee not found.

SAMPLE RUN

```
------ Employee Information ------

Employee 1:
Employee ID   : EMP101
```

```
Name          : AHMED
Department    : FINANCE
Salary        : 7500.00
Employee Ref  : FI01

Employee 2:
Employee ID   : EMP205
Name          : SARA
Department    : HR
Salary        : 6800.50
Employee Ref  : HR05

Employee 3:
Employee ID   : EMP310
Name          : OMAR
Department    : IT
Salary        : 8200.75
Employee Ref  : IT10

--------------------------------

Total number of created employees: 3

Enter new salary for the second employee:
7000.00

Updated Employee Information (Second Employee Only):
Name   : SARA
Salary : 7000.00

Enter Employee ID to search:
EMP205

Employee found.
Employee ID   : EMP205
Name          : SARA
Department    : HR
Salary        : 7000.00
Employee Ref  : HR05
```